Project Proposal for Auron

By Wong Chang Sheng

Cohort 13, Sigma School

1.0 Project Overview

1.1 Project Summary and Target Demographic

Auron is a budget planner where a user can track daily spendings of a set budget. A user can set how much spare money they have for the month and Auron will automatically set a daily budget plan for the rest of the moth (or alternatively, manually set by the user) and start tracking.

Auron also features multiple tools to assist with maintaining good impulse control such as rewards and accountability. One of the core features include a wishlist, where users can set desired items to buy in the future, and for each day that passes without using the entire budget, it gets saved up for the user buy them. Another feature includes an "accountability buddy" system where friends can keep an eye on your progress and reach out if something's amiss.

These are only some of the many features that Auron offers. This application is targeted towards people who have struggled with impulse purchases and want to make a step to improving themselves. One of the effective solutions for this (which is what Auron aims to do) is to visualize how much a person has spent so that they can fully comprehend and know how much money has been wasted.

2.0 Core Features

2.1 Monthly allowance and daily budget

On registration, the user sets up in two steps:

1. Set a monthly allowance
2. Set the daily budget, either:
    - Calculated automatically by splitting the allowance evenly across the remaining days of the month.
    - Typed in directly.

A user who registers mid-month gets the automatic split across the remaining days only. A typed-in daily budget is blocked if, over the remaining days, it adds up to more than the allowance.

Each expense logged has a name, a category (food, entertainment, transportation and so on) and an amount.

Overspending

The user picks one default for what happens when a day's budget goes below 0. Any overspend breaks the streak (2.6).

Option	Effect	Limits
1. Subtract from the next day	The overspent amount is taken from tomorrow's budget, and from later days if it is larger than one day's budget.	Debt that runs past the end of the month moves to next month's allowance.
2. Spread across the rest of the month	The overspent amount is divided across every remaining day's budget.	Disabled on the last day of the month.

Overspending past the monthly budget will eat into the next month’s budget (ie. Overspending on the last day, blowing the entire budget somewhere in the month, etc). If that wipes out next month's allowance too, the debt carries into the month after, and so on.

2.2 Wishlist

Unspent daily budget builds up in a wishlist pool, which the user spends on items they've chosen as rewards. Income can also be added to the pool (2.3).

Field	Required	Notes
Item name	Yes	
Price	Yes	In the user’s default currency
Desirability	Yes	A label from "Somewhat interested" to "Top priority"
Description	No	
Image	No	
Product link	No	

When the pool reaches or exceeds an item's price, the user is notified that they can buy it. A purchase deducts the price from the wishlist pool. It does not touch the daily budget and is not logged as an expense.

2.3 Income distribution

Additional income (such as sales, salary, refunds, etc) goes to one of three places the user chooses.

Option	Effect	Limits
1. Spread across the rest of the month	Divided across every remaining day's budget	Disabled on the last day of the month
2. Add to today only	Added to today's budget. Any part left unspent flows to the wishlist pool	None

3. Add to the wishlist pool	Added straight to the pool	None

2.4 Statistics

A dedicated page tracks the user's saving journey with:

- Weekly or monthly charts of how close each day came to using up its budget.
- A breakdown of which categories the user spends on most.
- Smaller statistics including:
    - Current and highest streaks
    - The percentage of the monthly allowance spent
    - Amount of money saved historically
    - If the user has spent more than they saved, or vice versa

2.5 Accountability buddy system

Users can search for other users and send a friend request, which stays pending until accepted. Once accepted, both friends see each other's:

- Username
- Streak
- Daily budget and each day’s spent amount
- Budget-usage charts

2.6 Streaks

A day adds to the streak when spending stays at or above 0 remaining. The streak breaks the moment spending goes below 0.

Miscellaneous notes

1. Reaching exactly 0 does not break the streak.
2. A day that starts at 0 because of carry-over debt still counts if the user spends nothing.

2.7 Spending states

The background colour changes as the day's budget is spent, with warnings at 75% and 90% used. A day that starts at 0 has no 75% or 90% state because it is already depleted.

2.8 AI helper

The user can choose to ask AI for insights on their current spendings. They can choose the period's worth of context for the AI to analyze, such as expenses for the current week, or the current month.

The backend will calculate the total expenses, the budget left, any days where the user has overspent, the user's streak and the top three top-spending categories.

The AI helper will take in this input, and give its own thoughts on the user's spending trend:

1. Praise: If any, the AI will congratulate or praise the user on their ample spending.
2. Concerns: If any, the AI will note on the user's overspending or habitual spending.
3. Suggestions: The AI will give suggestion on the user's next actions.

As a precautionary measure, before the user uses the AI for the first time, a consent screen will appear. This means that it will need

2.9 Month boundaries and edit rules

Situation	Rule
New month starts	The user is prompted for a monthly allowance and daily budget, unless they turned on auto-reuse of the previous month's. Any carried-over overspend is deducted from the allowance first.
Overspend debt runs past month end	Deducted from next month's allowance, chaining into later months if needed.
Last day of the month	The options for "spreading the extra cost/income across the entire month" when overspending or getting income respectively are disabled. Overspending is deducted from next month's allowance.
User registers mid-month	The automatic daily budget split covers the remaining days only.
Editing past expenses	Expenses can be edited or deleted before the day ends, after that they are locked.
Edit would make the wishlist pool negative	The edit is blocked. This happens when the leftover money it changes has already been spent on a wishlist item.

2.10 Currency conversion

Users can pick a currency wherever money is entered or shown, with RM as the default. Rates come from the free Frankfurter API.

The currency picker appears on:

- Monthly allowance
- Daily budget
- Remaining budget for the day
- Daily expenses
- Income
- Wishlist item prices
- A dedicated currency conversion page

A free external API will be used called Frankfurter (https://frankfurter.dev/) to fetch the latest exchange rates for the day. If there are missing exchange rates for the day, it will use the most recent past rate.

Additionally, upon the first setup of Auron after registration, the user will be prompted to choose a currency to use as default. This can be changed in the settings, however there will be a warning that changing it will cause all data in the old currency to be updated to the new one at today's rate.

Miscellaneous notes

1. The app will default to RM.
2. For each of the features that include currency conversion, they will show both the converted and the original RM amount.

2.11 Flexible options

Auron offers flexible options to fine tune how each user wants to maintain their finances or set their preferences. For instance, when deducting from the daily budget, the user may choose to deduct from the current day, or redistribute the cost across the rest of the month. Alternatively, if the user wants to add income, they can either choose add it to the current day, redistribute it across the rest of the month, or add it to the wishlist pool.

2.12 Responsive layout

Auron features separate layouts for desktop and mobile views. For mobile viewers, it will show a differently designed layout, but will stay intuitive to use and Auron is especially useful when used on the go.

2.13 Notifications

Auron features a notification system that sits at the top of the page, which will show notifications based on:

1. Whether your wishlist pool has reached or exceeded the price of your wishlist items
2. Your streak at specific milestones, such as 7 days, 14 days, 30 days, etc.

3. Whether your friends has overspent

3.0 Entity Relationships

3.1 “users” table

Holds each user’s profile and streak.

1. id: UUID, primary key, linked to auth.users.id
2. username: VARCHAR(50), unique (auto-generated from email)
3. display_name: VARCHAR(50), nullable
    a. Can be set by the user to appear as the name they want
4. current_streak: INTEGER, default 0
5. highest_streak: INTEGER, default 0
6. ai_consent_at: TIMESTAMPTZ, nullable
    a. When using the AI helper, it needs to access the user’s expense and income data, so there will be a consent screen for the user to accept or decline before using it
7. created_at and updated_at timestamps

3.2 “monthly_plans” table

Holds each month’s allowance and how it is split into daily budgets.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. month: DATE
    a. Always the first day of the month (e.g. 2026-10-01). Unique together with user_id, so a user has one plan per month.
4. allowance_input_amount: NUMERIC(12,2)
    a. The allowance exactly as the user typed it, in the currency they picked.
5. allowance_input_currency: CHAR(3)
6. allowance_in_default_currency: NUMERIC(12,2)
    a. The allowance converted to default currency. All budget calculations use this value.
7. carry_in_debt: NUMERIC(12,2), default 0
    a. Overspending the previous month couldn’t absorb. It is deducted from the allowance before the allowance is split into days.
8. carry_out_debt: NUMERIC(12,2), default 0
    a. Overspending the current month couldn’t absorb. It becomes next month’s carry_in_debt.
9. budget_mode: ENUM(‘auto’, ‘manual’)
    a. auto: the monthly allowance (after carry_in_debt) is divided evenly across the remaining days.
    b. manual: the user manually inputs a daily amount.

10. daily_amount: NUMERIC(12,2)
    a. The daily budget.
11. created_at and updated_at timestamps

3.3 “daily_budgets” table

Holds one row per user per day, with that day’s budget and spending.

1. id: UUID, primary key
2. plan_id: UUID, foreign key → monthly_plans.id
3. user_id: UUID, foreign key → users.id
    a. Duplicated from the plan so row-level security and per-user queries don’t need a join.
4. date: DATE, unique together with user_id
5. adjustment_total: NUMERIC(12,2), default 0
    a. The net effect on this day of spread expenses, overspending carried from earlier days, and income added to it.
6. spent_total: NUMERIC(12,2), default 0
    a. The total of expenses deducted from this day. The remaining budget is daily_amount (from monthly_plans table) + adjustment_total − spent_total.
7. streak_kept: BOOLEAN, nullable
    a. Null while the day is still ongoing, when the day ends, set to true if the remaining daily budget is RM0 or more
8. created_at and updated_at timestamps

3.4 “user_settings” table

Hold each user’s settings

1. user_id: UUID, primary key, foreign key → users.id
    a. One row per user, created by the same registration trigger as “users”.
2. default_deduction: ENUM(‘today’, ‘spread’), default ‘today’
    a. Deduct expenses from today, or spread the cost across the remaining days of the month.
3. monthly_setup: ENUM(‘reuse_previous’, ‘always_prompt’), default ‘always_prompt’
    a. Copy last month’s allowance and daily budget if there is one. Otherwise the user is prompted.
4. default_overspend: ENUM(‘next_day’, ‘spread’), default ‘next_day’
    a. The overspend options, to either subtract from the next day’s budget or spread the cost across all remaining days
5. default_income: ENUM(‘today’, ‘spread’, ‘wishlist’), default ‘today’

    a. The income options, either add the income to today's budget, spread it across the remaining days of the month or add it to the wishlist pool
6. default_currency: CHAR(3), default 'MYR'
    a. The default currency for the user.
    b. There will be a warning that when changing the default currency midway, all records will be converted into that new default currency at today's rate.
7. timezone: VARCHAR(50), default 'Asia/Kuala_Lumpur'
    a. Decides when the user's day ends and the day is closed.
8. updated_at timestamp

3.5 "categories" table

Holds the fixed list of expense categories

1. id: SMALLINT, primary key
2. name: VARCHAR(50), unique
    a. For example, Food, Transportation, Entertainment.

3.6 "expenses" table

Holds every expense the user logs.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. day_id: UUID, foreign key → daily_budgets.id
    a. The day the expense was made on.
4. category_id: SMALLINT, foreign key → categories.id
5. name: VARCHAR(100)
6. input_amount: NUMERIC(12,2)
    a. The amount exactly as entered in the selected currency, may or may not be MYR.
7. input_currency: CHAR(3)
    a. The currency used for the input.
8. amount_in_default_currency: NUMERIC(12,2)
    a. input_amount × default currency exchange rate, used to display the budget in the user's default currency.
9. deduction_option: ENUM('today', 'spread')
    a. Pre-filled from user_settings.default_deduction; the user can change it per expense.
10. overspend_option: ENUM('next_day', 'spread')
    a. Pre-filled from user_settings.default_overspend. Only used if this expense takes the day below 0. On the last day of the month, any excess goes to next month's allowance whichever option is set.

11. created_at and updated_at timestamps
12. deleted_at: TIMESTAMPTZ, nullable
    a. Deletion only marks the row instead of removing it. recalc_days() ignores marked rows. Edits and deletes are allowed until one day after the expense's day has passed; after that the row is locked.

3.7 "incomes" table

Holds additional income.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. day_id: UUID, foreign key → daily_budgets.id
    a. The day the income was received.
4. note: VARCHAR(100), nullable
    a. An optional label, e.g. "Refund", "Sales", "Salary", etc.
5. input_amount: NUMERIC(12,2)
    a. The amount exactly as entered in the selected currency, may or may not be MYR.
6. input_currency: CHAR(3)
    a. The currency used for the input.
7. amount_in_default_currency: NUMERIC(12,2)
    a. input_amount × default currency exchange rate, used to display the budget in the user's default currency.
8. distribution: ENUM('today', 'spread', 'wishlist')
    a. Pre-filled from user_settings.default_income. today adds it to that day's budget. spread divides it across the remaining days and isn't offered on the last day of the month. wishlist adds it straight to the pool.
9. created_at and updated_at timestamps

3.8 "wishlist_items" table

Holds the items the user is saving up for as rewards.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. name: VARCHAR(100)
4. description: TEXT, nullable
5. input_amount: NUMERIC(12,2)
    a. The amount exactly as entered in the selected currency, may or may not be MYR.
6. input_currency: CHAR(3)
    a. The selected currency for the input
7. image_url: TEXT, nullable

    a. Path to the uploaded image in Supabase Storage.
8. product_link: TEXT, nullable
9. desirability: SMALLINT
    a. A label from somewhat interested to top priority
10. affordable_notified: BOOLEAN, default false
    a. Set to true when the wishlist pool reaches or exceeds the current item's price.
11. purchased_at: TIMESTAMPTZ, nullable
    a. Not null means the item has been bought.
12. purchase_amount_in_default_currency: NUMERIC(12,2)
    a. Stores the purchase amount in the default currency.
13. created_at and updated_at timestamps

3.9 "friendships" table

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
    a. The user who sent the request.
3. friend_id: UUID, foreign key → users.id
    a. The user who received it
4. status: ENUM('pending', 'accepted', 'declined'), DEFAULT 'pending'
    a. Once a request is accepted, row-level security lets both users see each other's data.
5. created_at and updated_at timestamps

3.10 "ai_insights" table

Holds each result the AI helper has produced. To save on tokens, the backend will handle calculations and the AI only has to inspect it and generate a response.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. period: ENUM('week', 'month')
   a. The period's worth of expense data for the AI helper to analyze
4. content: JSONB
   a. Contains a JSON structure of the response
    1. Praise: The AI's response that notes the user's ample spending
    2. Concerns: The AI's response on the user's habitual spendings or overspending
    3. Suggestions: Next actions for the user to take
5. created_at: TIMESTAMPTZ

3.11 “notifications” table

Holds the in-app notification list.

1. id: UUID, primary key
2. user_id: UUID, foreign key → users.id
3. type: ENUM(‘budget_75’, ‘budget_90’, ‘wishlist_affordable’, ‘friend_request’)
4. payload: JSONB
    a. Details for that type, e.g. the item’s id and name, or the requester’s username.
5. read_at: TIMESTAMPTZ, nullable
    a. Gets set when the user reads the notification
6. created_at timestamp

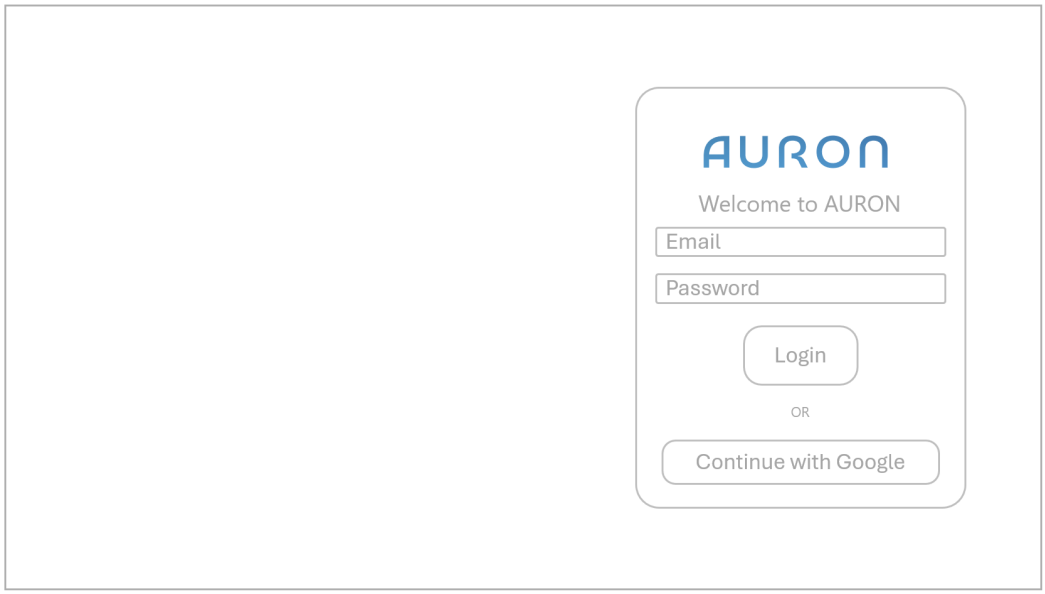
3.12 “exchange_rates” table

Holds the latest exchange rate for every currency which are fetched via cronjob every day.

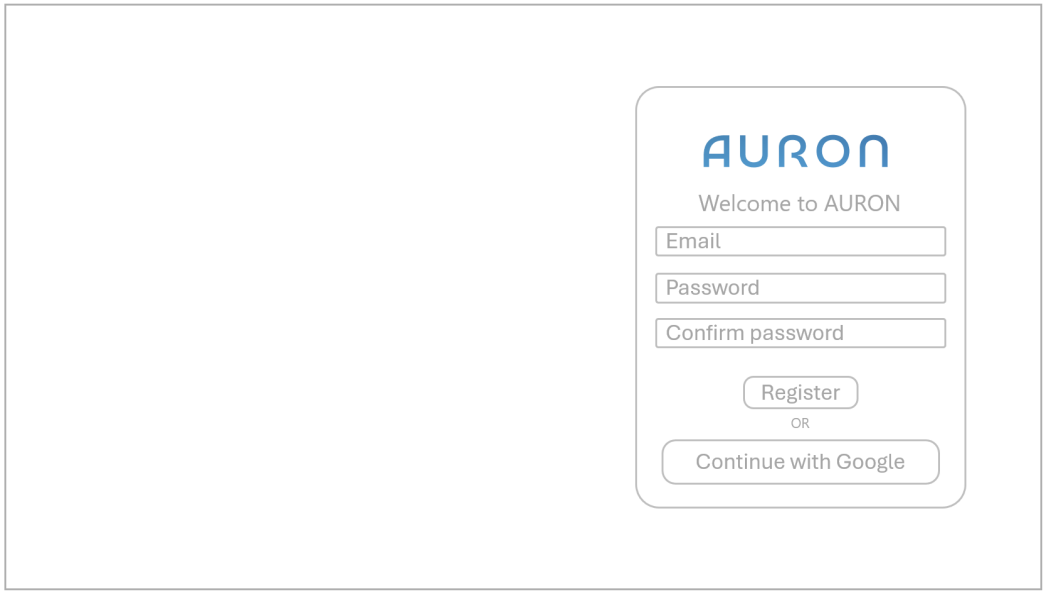
1. currency: CHAR(3), primary key
2. rate: NUMERIC(18,8)
    a. 1 MYR = rate units of currency. Conversions between two other currencies divide one rate by the other, e.g. USD → SGD = rate(SGD) ÷ rate(USD).
3. rate_date: DATE
    a. The date the API published the rate. It’s shown to users when rates are out of date.
4. fetched_at: TIMESTAMPTZ

4.0 Wireframes

Login



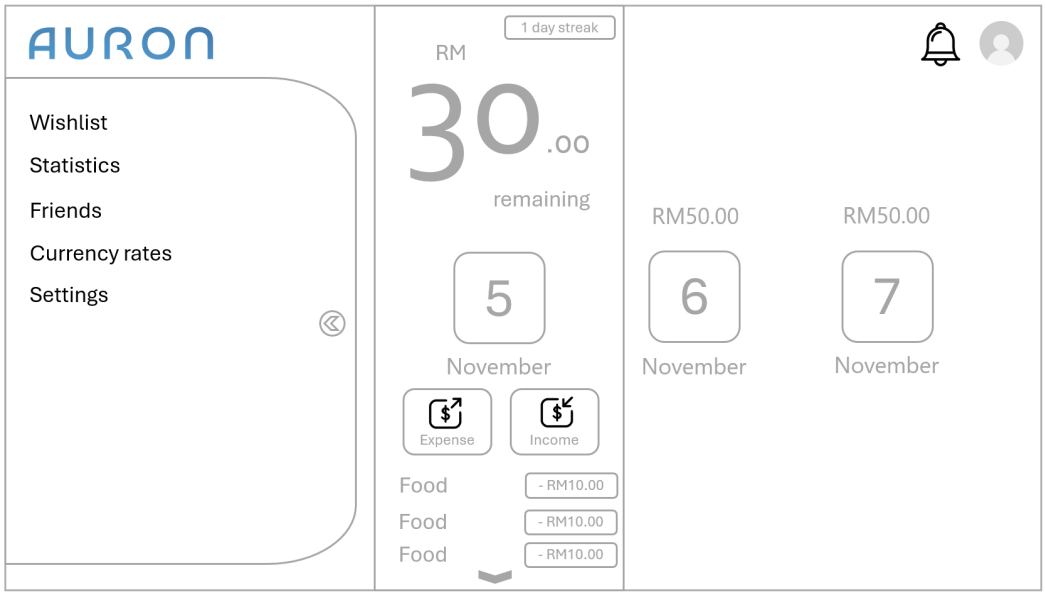
Registration



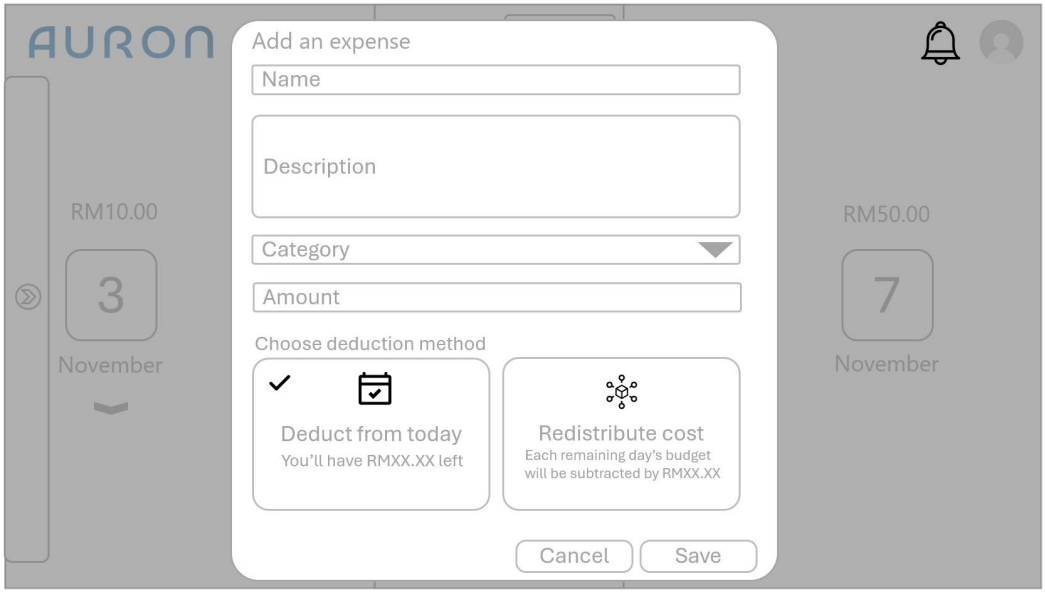
Main page



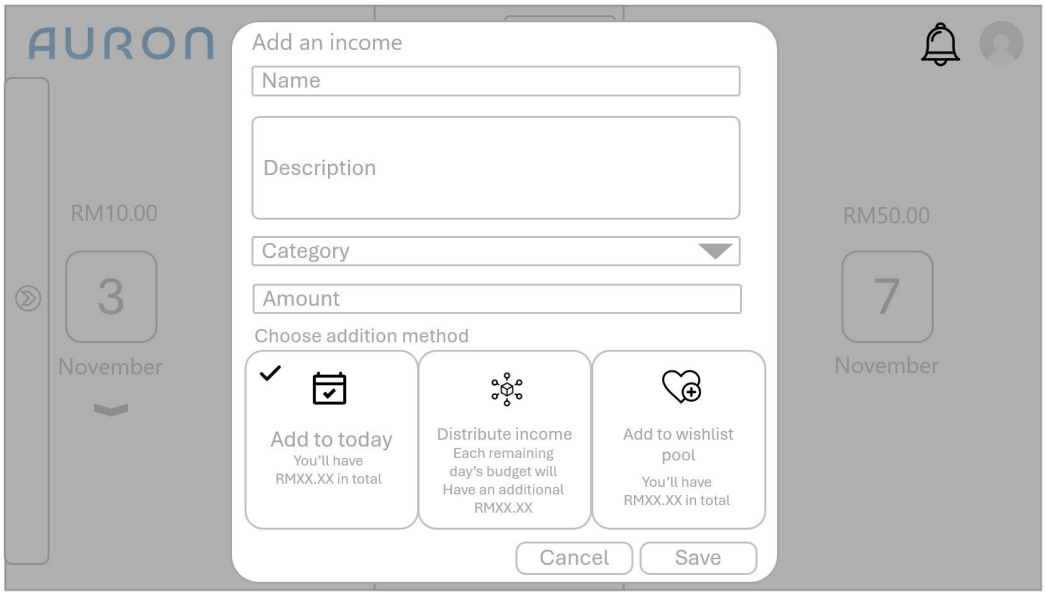
Sidebar



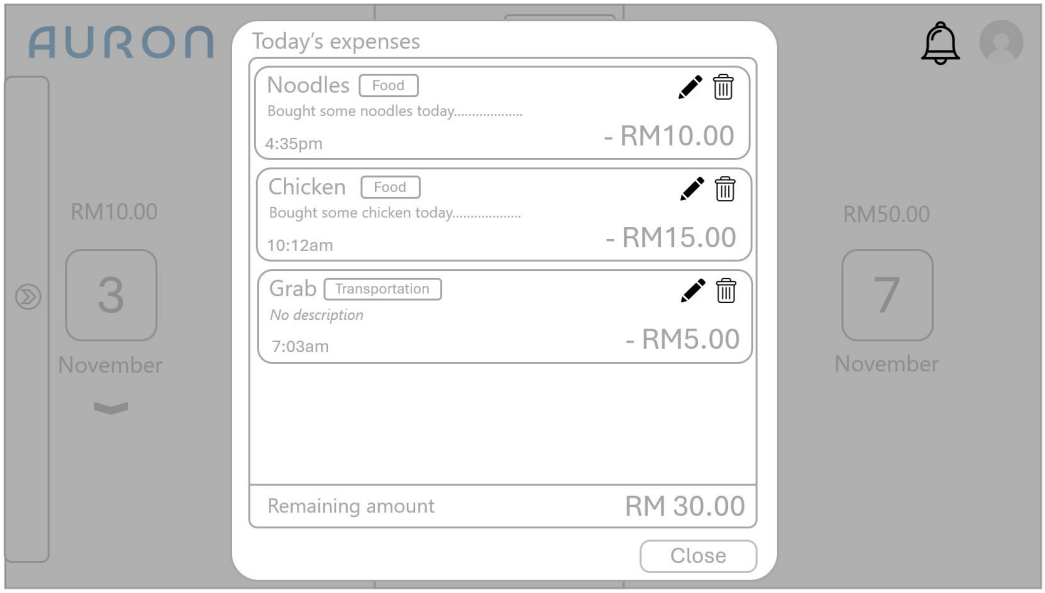
Expenses



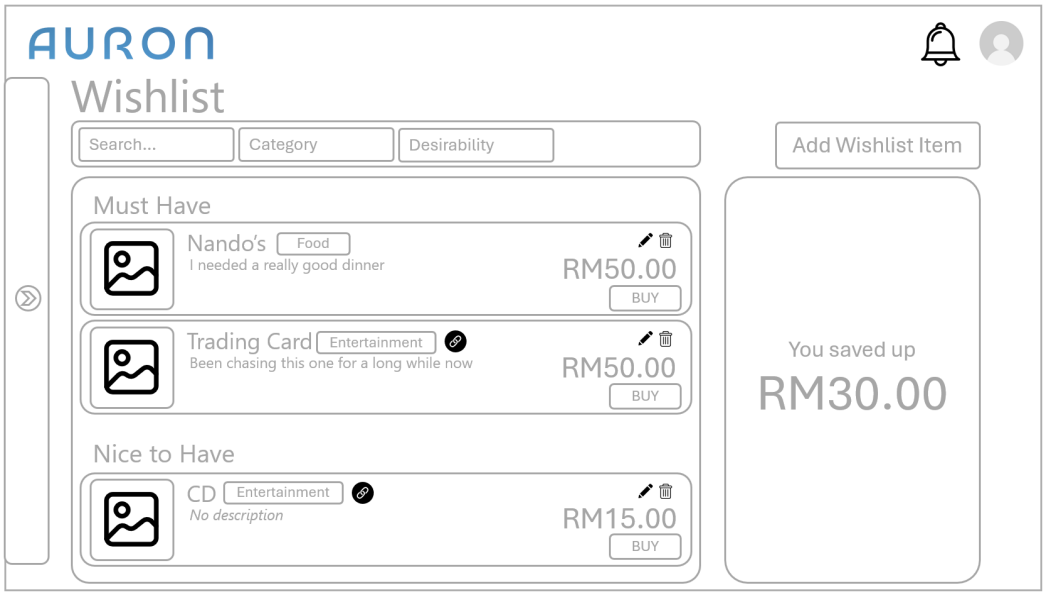
Income



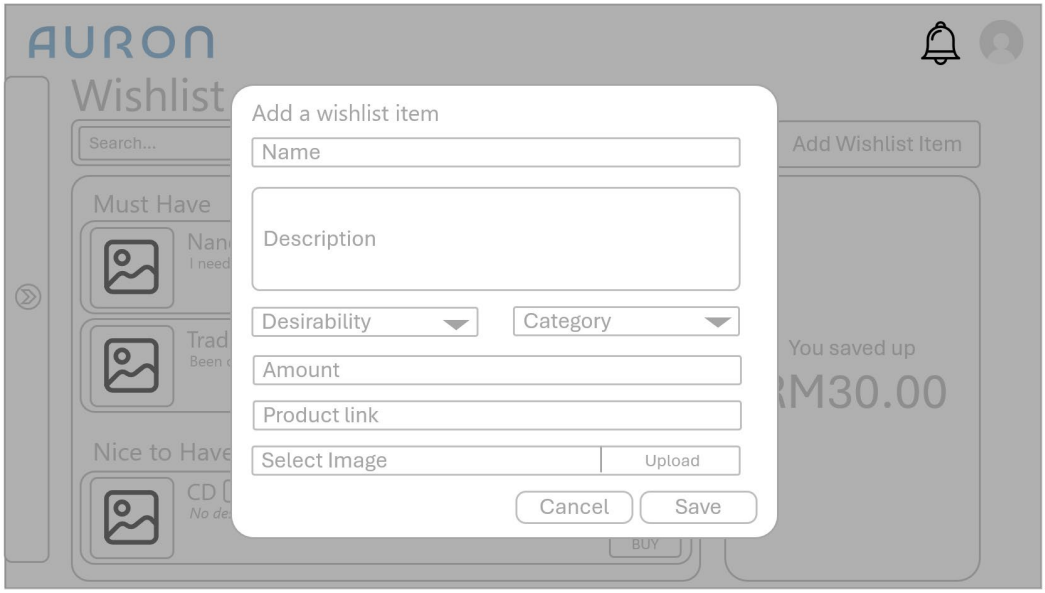
View expenses



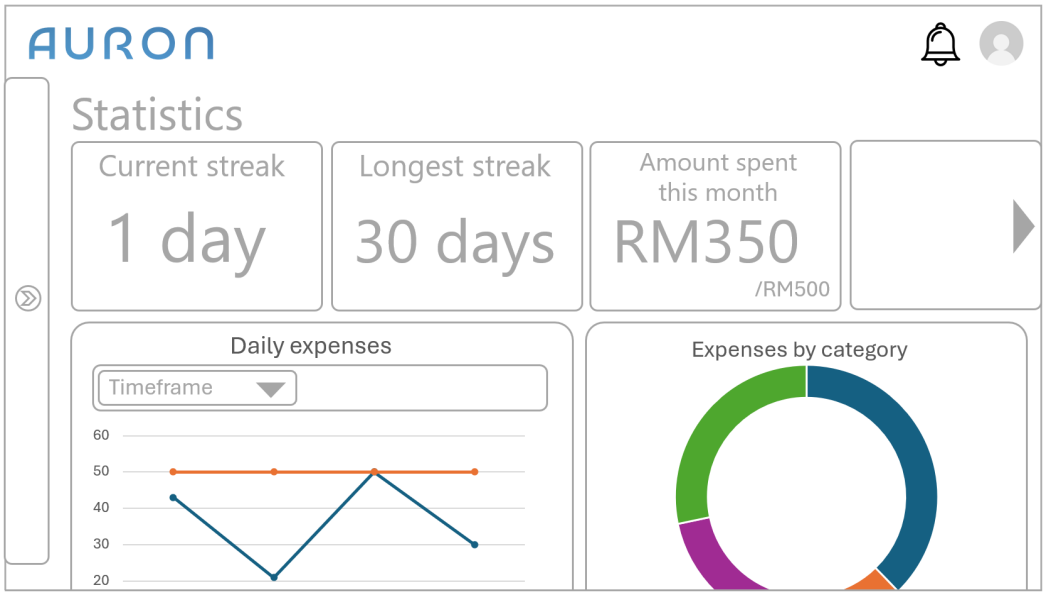
Wishlist



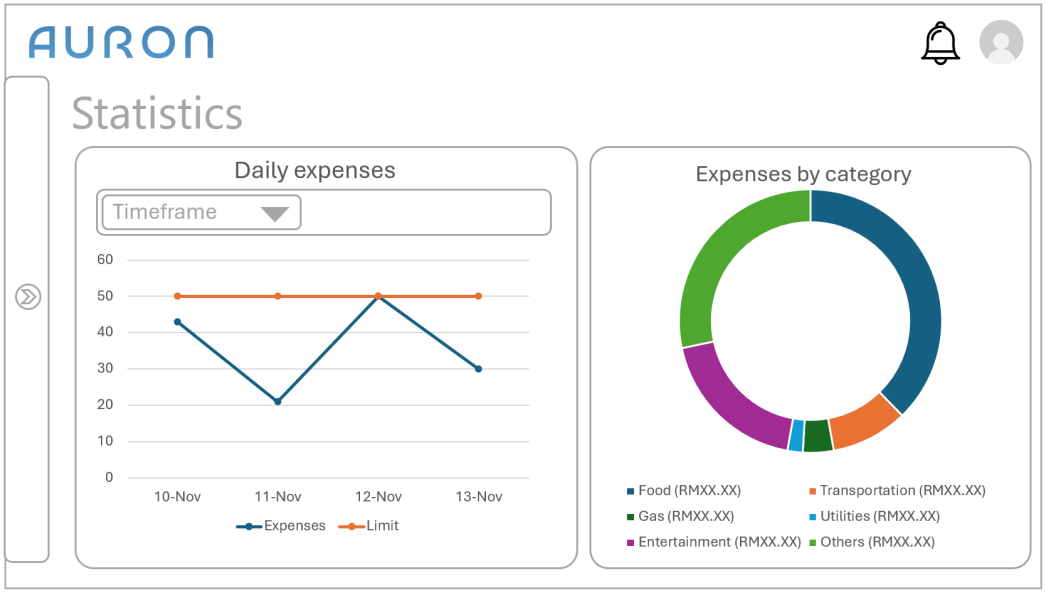
Add Wishlist Item



Statistics



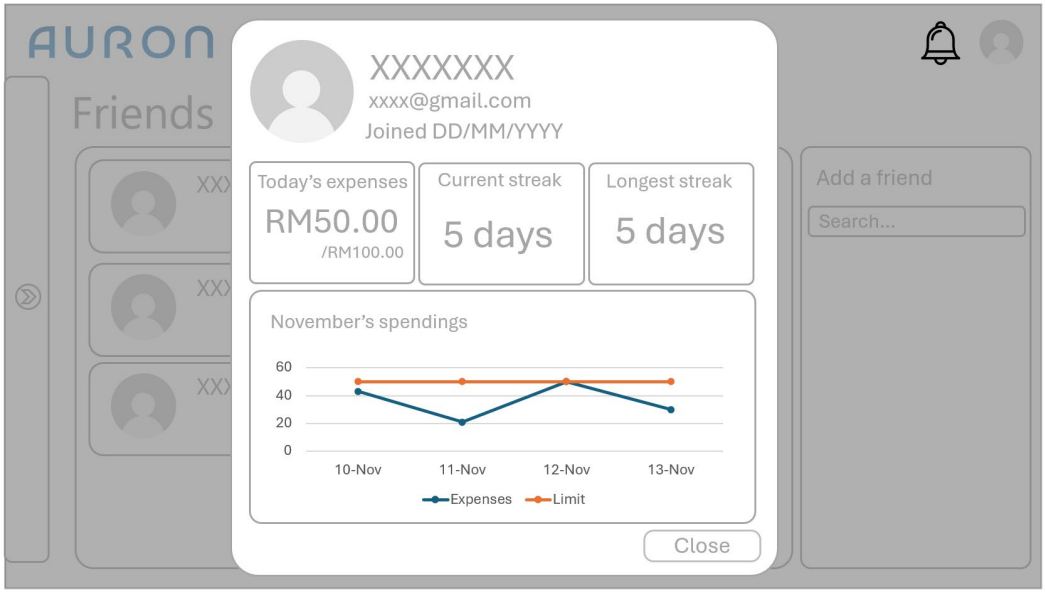
Statistics



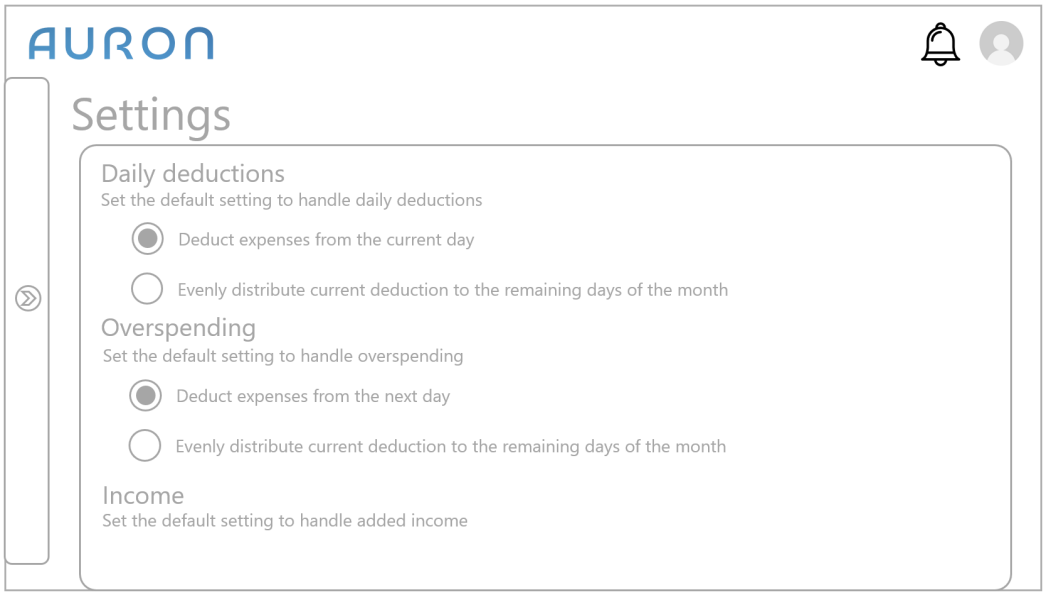
Friends



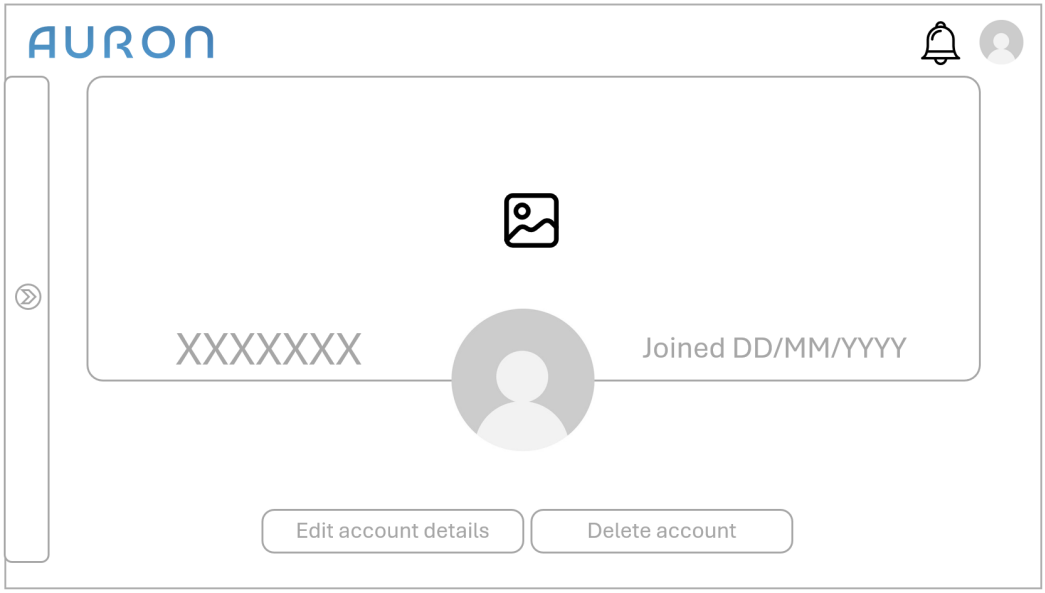
Friends



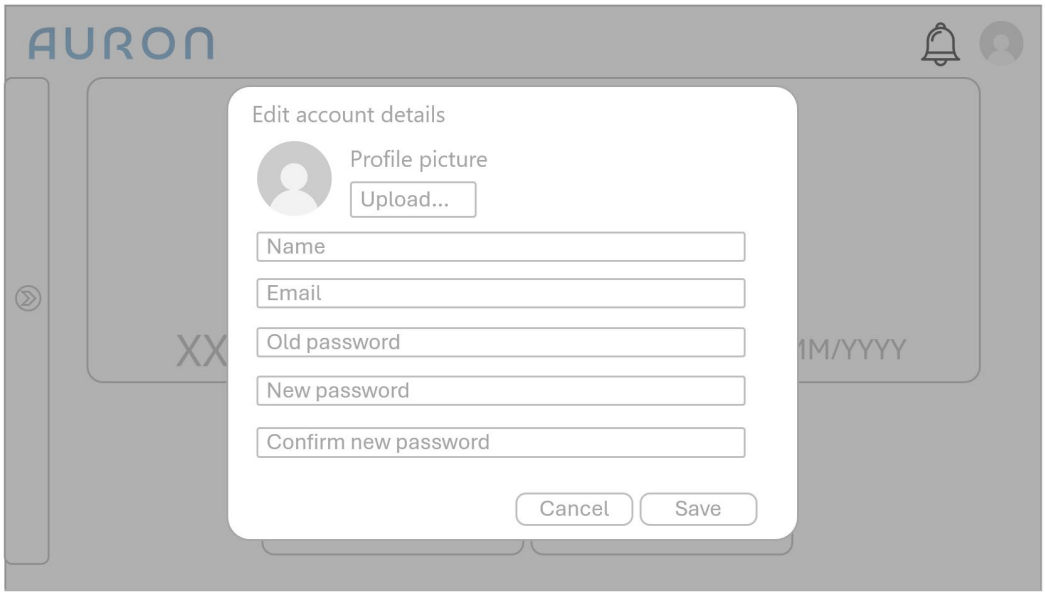
Settings



Profile

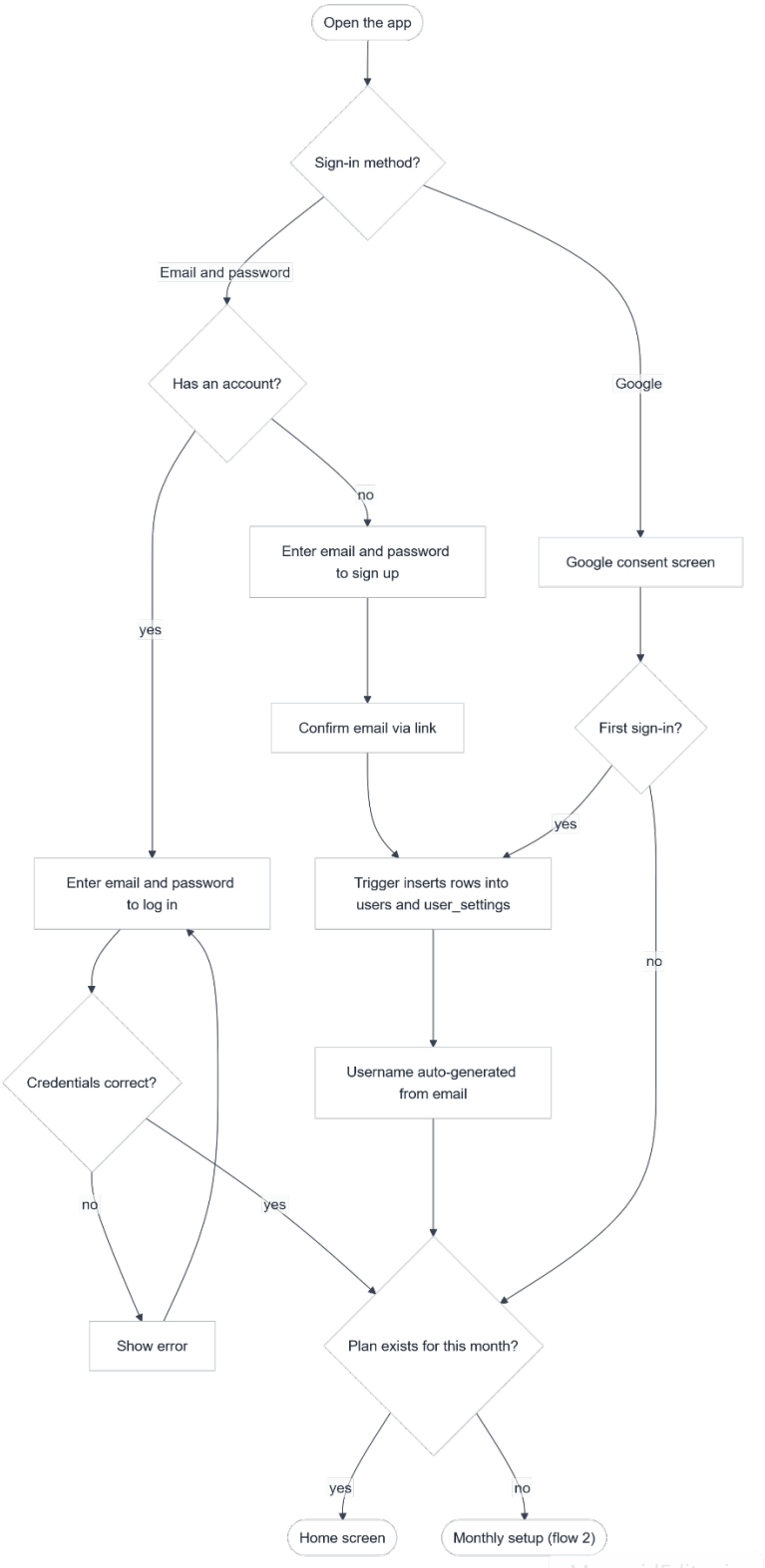


Edit Profile

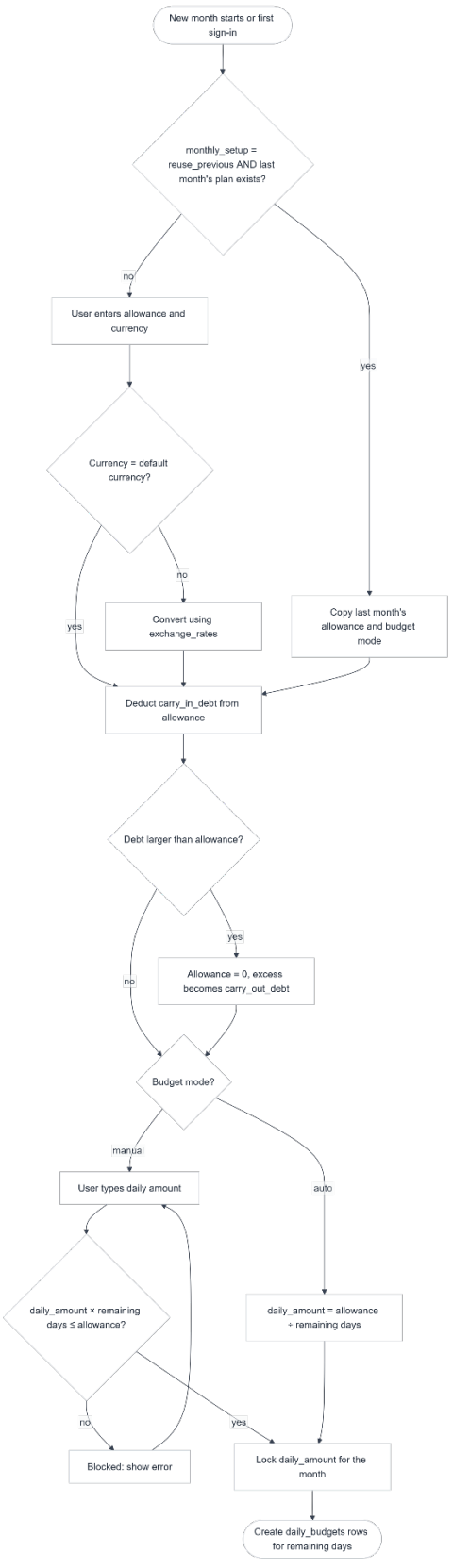


5.0 Process Flows

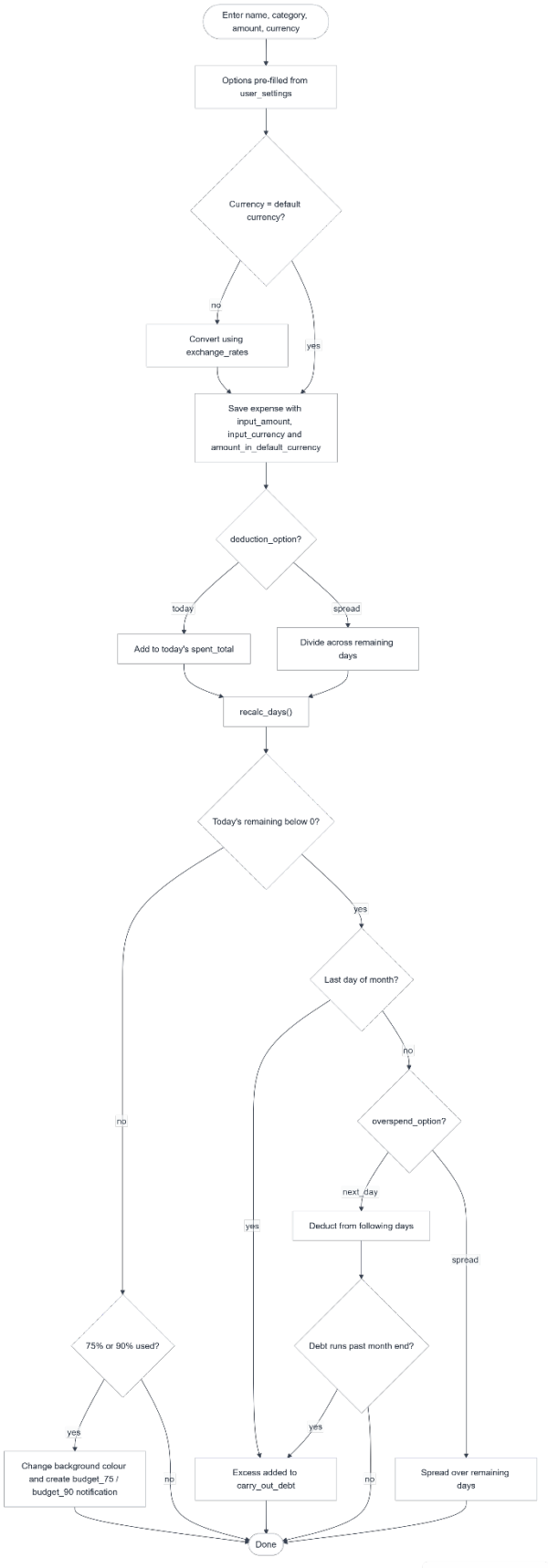
5.1 Login and registration



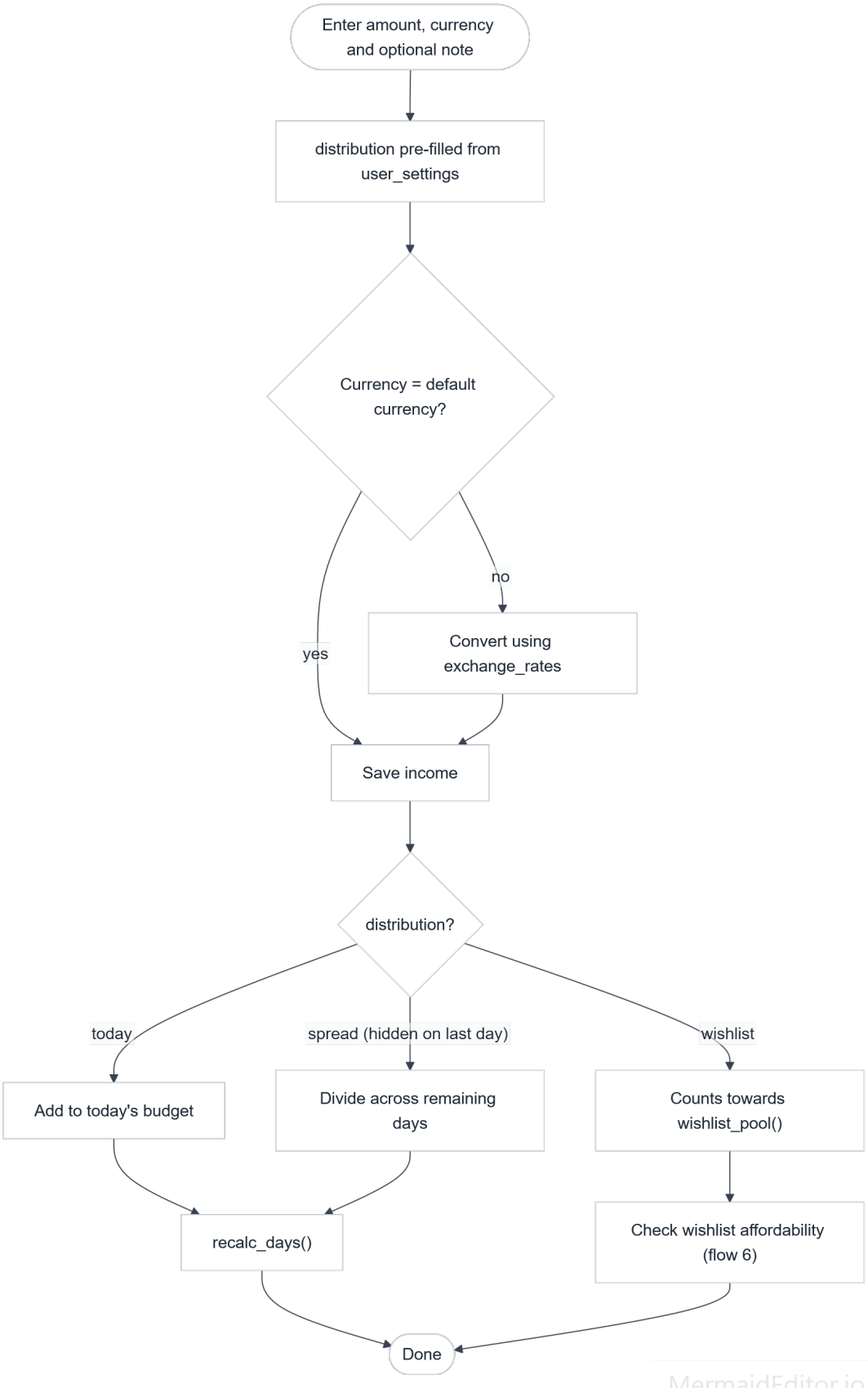
5.2 Monthly budget setup



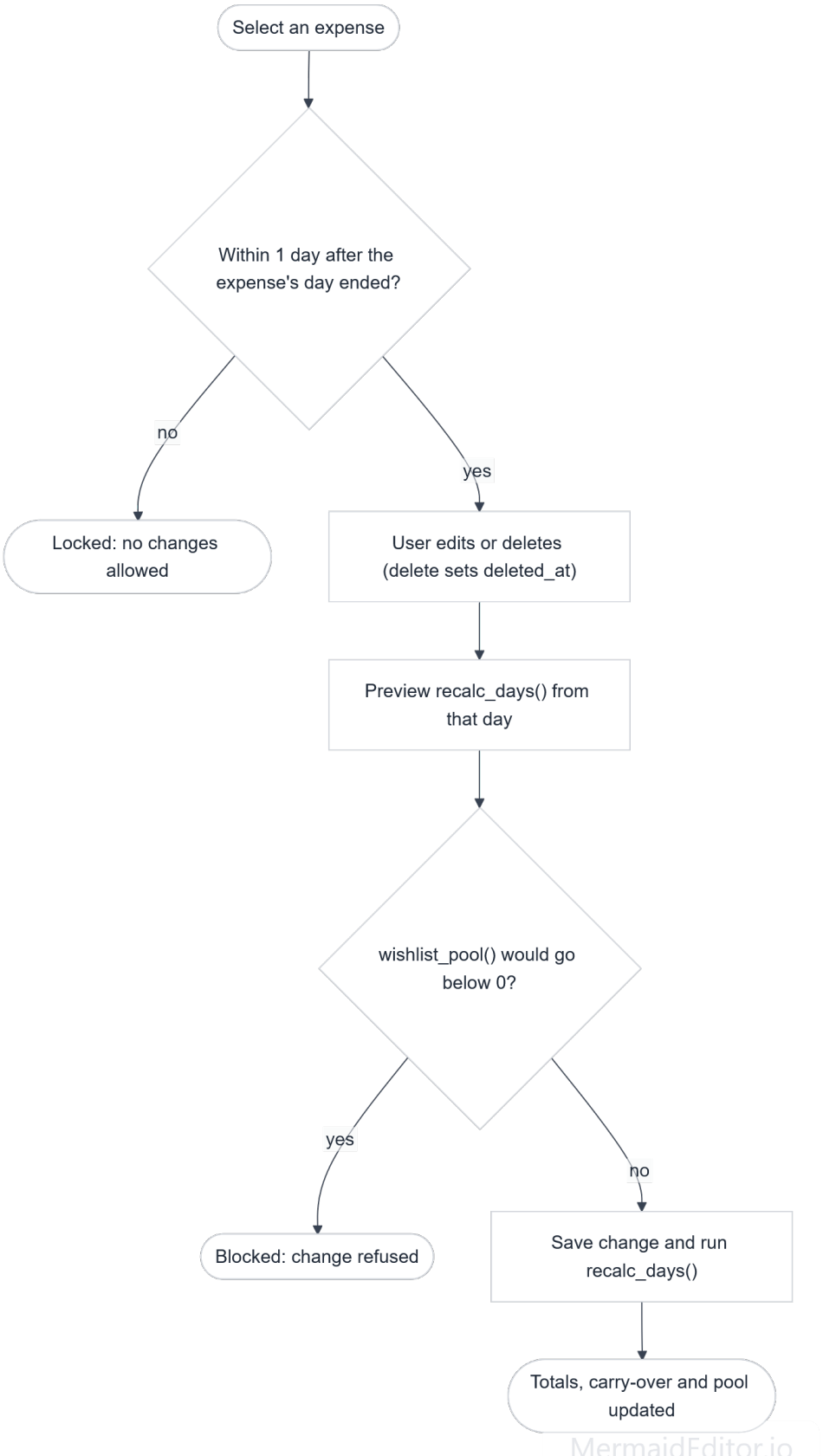
5.3 Expense logging



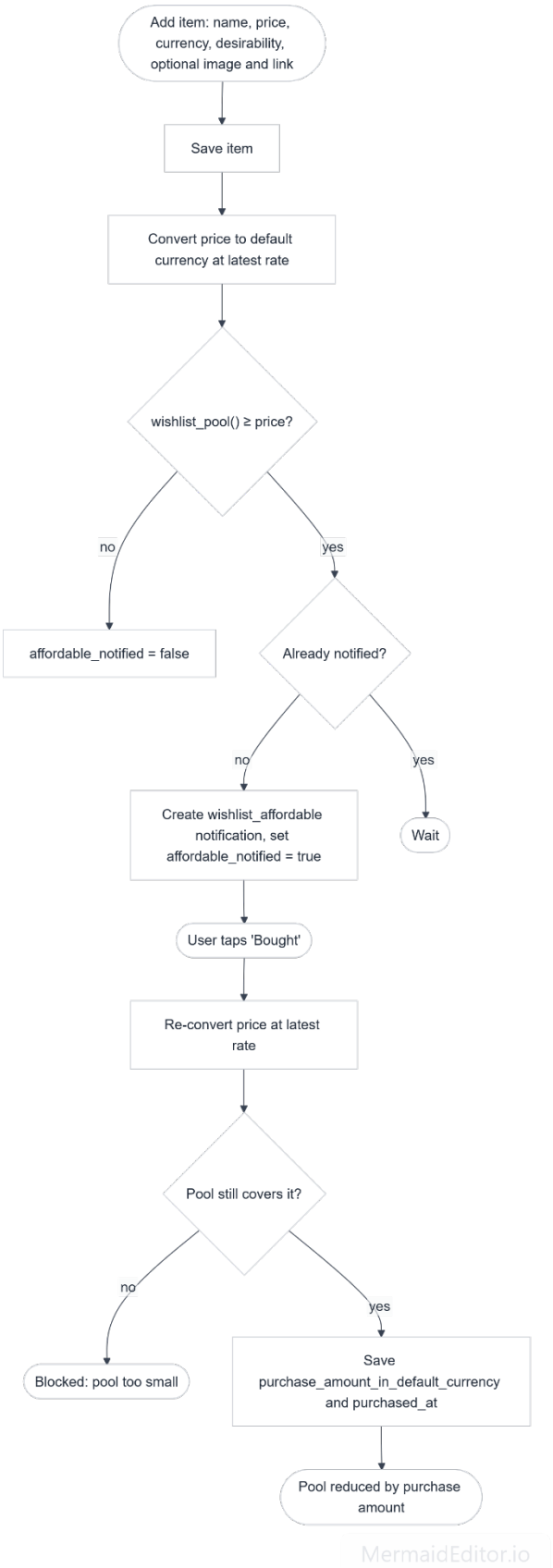
5.4 Income logging



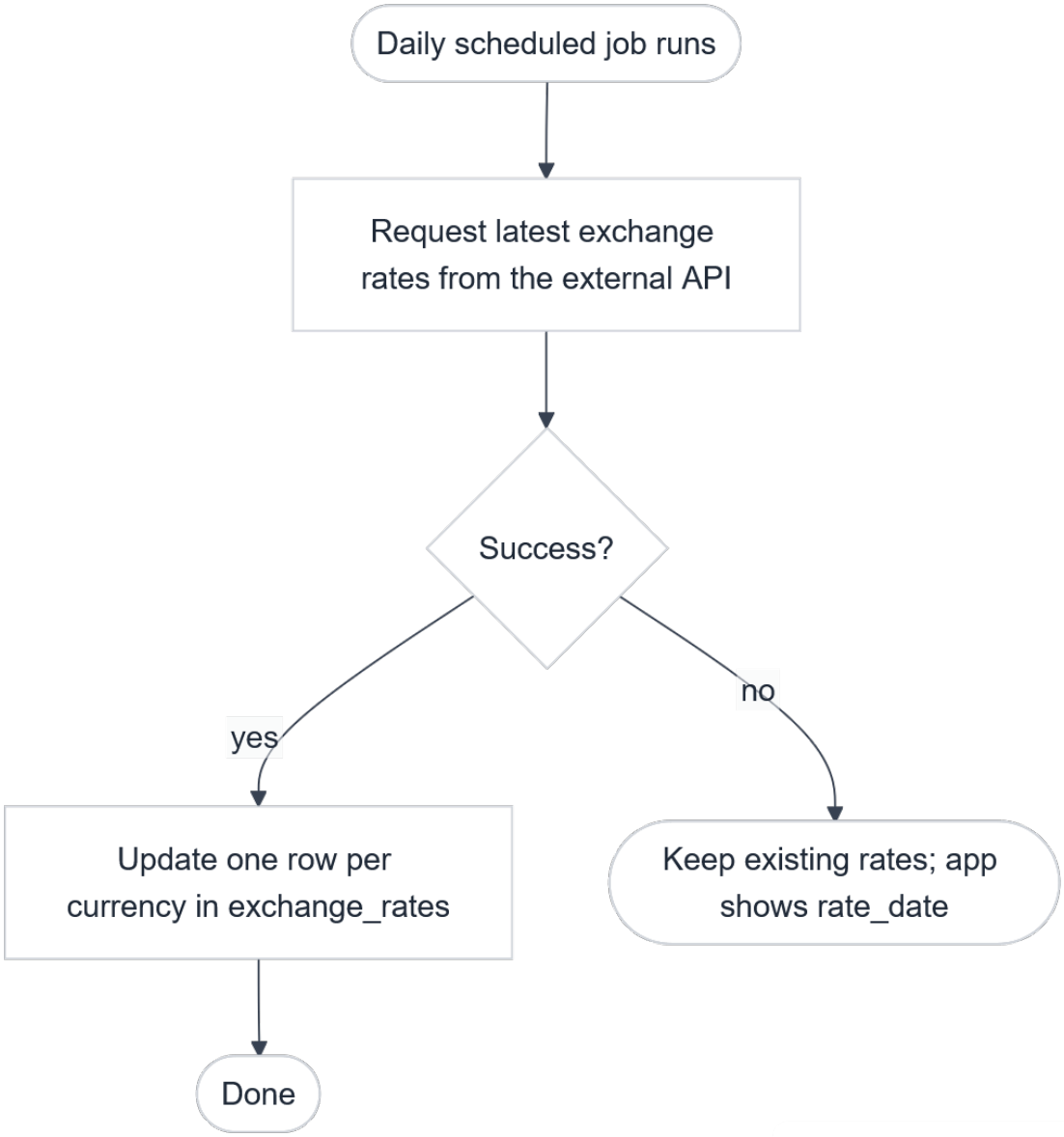
5.5 Editing/deleting expense



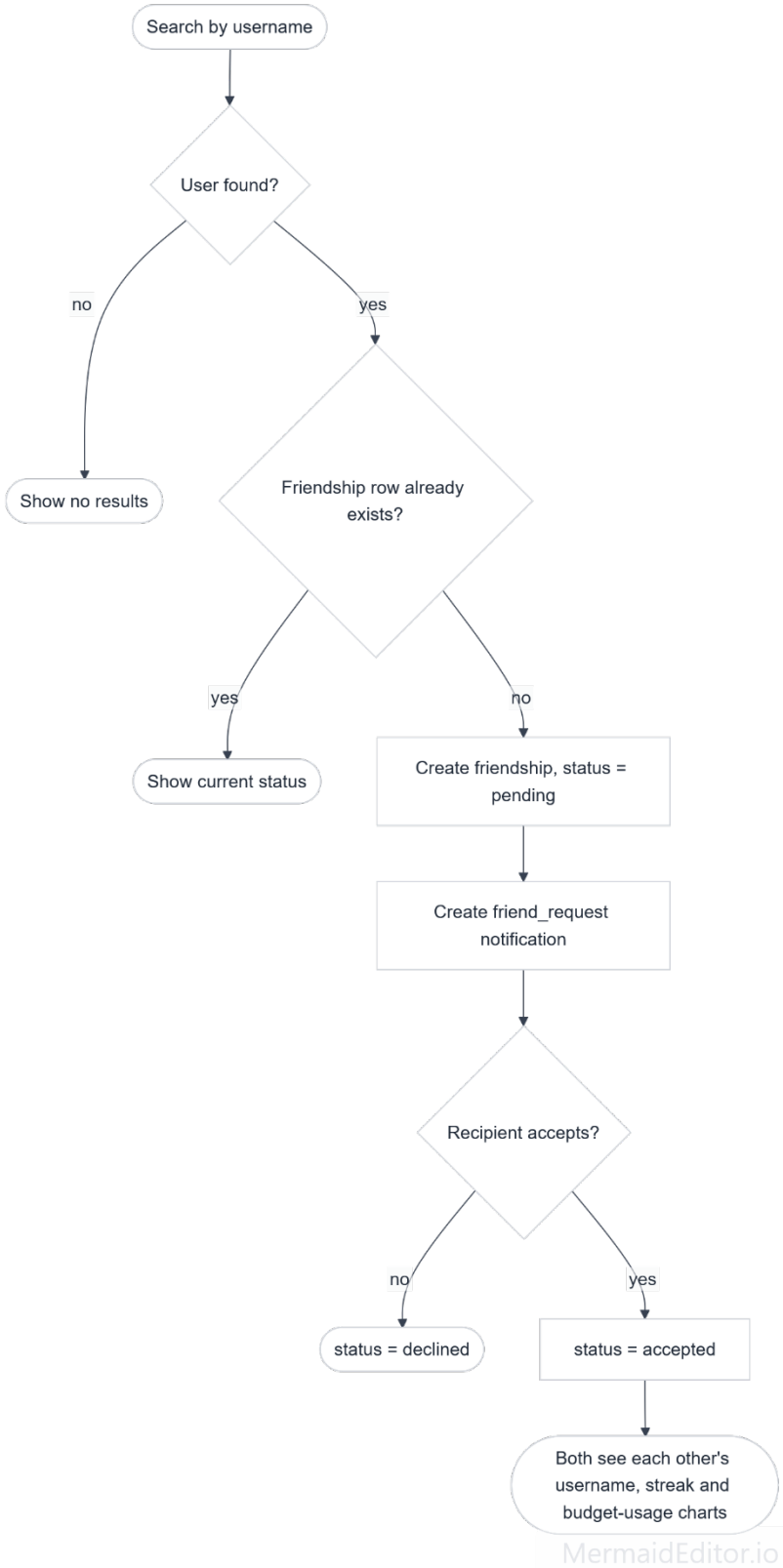
5.6 Wishlist


Add item: name, price, currency, desirability, optional image and link
Save item
Convert price to default currency at latest rate
wishlist_pool() ≥ price?
no
yes
affordable_notified = false
Already notified?
no
yes
Create wishlist_affordable notification, set affordable_notified = true
Wait
User taps 'Bought'
Re-convert price at latest rate
Pool still covers it?
no
yes
Blocked: pool too small
Save purchase_amount_in_default_currency and purchased_at
Pool reduced by purchase amount
MermaidEditor.io

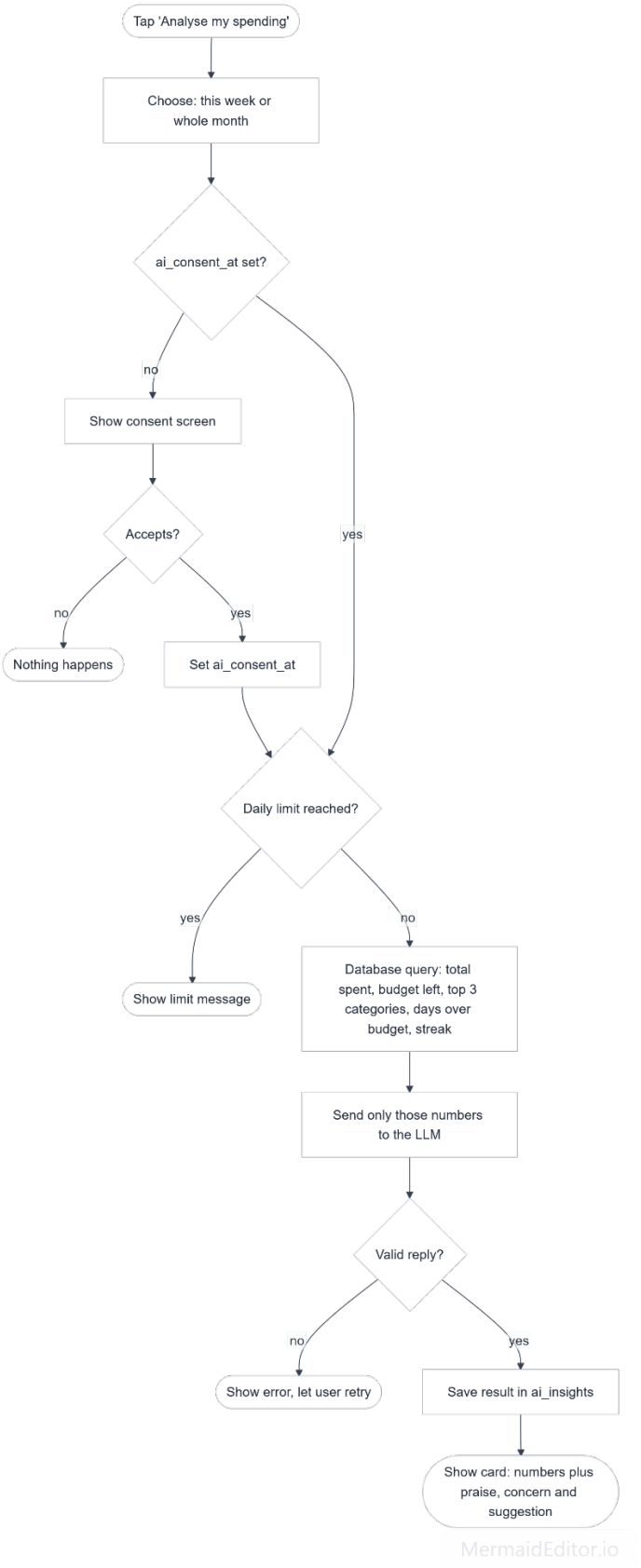
5.7 Exchange rates



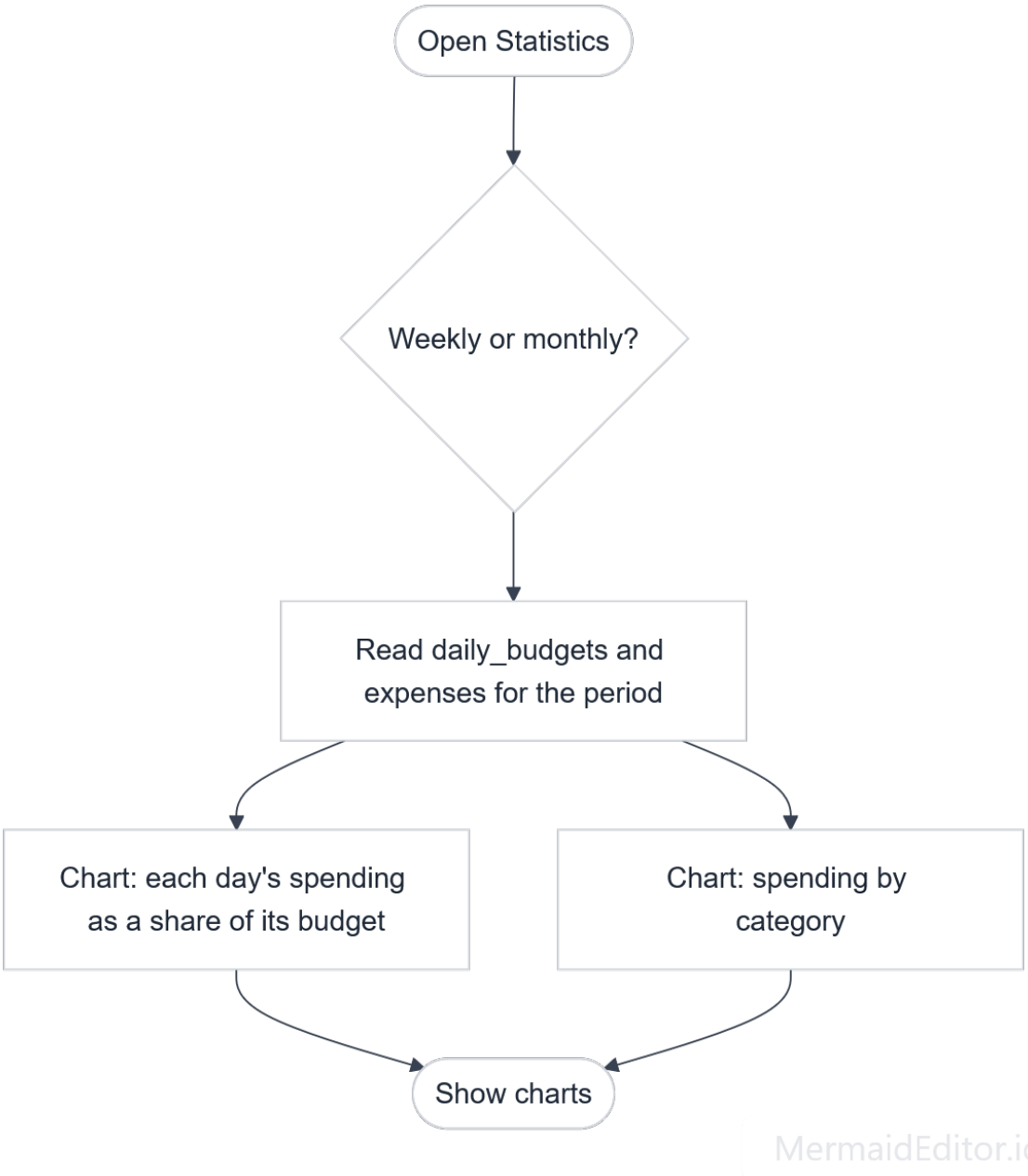
5.8 Friends


Search by username
User found?
no
yes
Show no results
Friendship row already exists?
yes
no
Show current status
Create friendship, status = pending
Create friend_request notification
Recipient accepts?
no
yes
status = declined
status = accepted
Both see each other's username, streak and budget-usage charts
MermaidEditor.io

5.9 AI helper


Tap 'Analyse my spending'
Choose: this week or whole month
ai_consent_at set?
no
yes
Show consent screen
Accepts?
no
yes
Nothing happens
Set ai_consent_at
Daily limit reached?
yes
no
Show limit message
Database query: total spent, budget left, top 3 categories, days over budget, streak
Send only those numbers to the LLM
Valid reply?
no
yes
Show error, let user retry
Save result in ai_insights
Show card: numbers plus praise, concern and suggestion
MermaidEditor.io

5.10 Statistics



6.0 Project Architecture and Developmental Process

6.1 Project Stack

Sector	Libraries or frameworks used
Frontend	React, Bootstrap, React Bits
Backend	JavaScript, Node, Express, Supabase Auth, Google OAuth
Database	PostgreSQL
Deployment and hosting	Supabase, Vercel, GitHub
External APIs	Groq (AI API), Frankfurter (Currency exchange rates)
Testing	Jest, Supertest, Vitest

6.2 Authentication

The user is able to create an account and log in as normal to the website, or they can choose to proceed with Google.

Login and registration will be handled by Supabase Auth and Google OAuth. The former makes logins and registrations easier to maintain as it comes with its own access and refresh token generation, password hashing, validation and more. The latter provides a more convenient option for users to access Auron.

6.3 Explored Options and Trade-offs

Budget planning

Option	Notes	Pros	Cons
Full budget planning	Would contain options to set your entire savings balance and mandatory spendings (eg. Rent, insurance, etc)	Even more detailed insights on the user's current financial situation	Out of scope, takes much longer to set up and contains a lot of different edge cases to handle across three different balance pools (savings, monthly allowance, daily budget)
Habit tracker	Only manage the user's monthly allowance and their daily budget	Easier setup, more focused on habit tracking and easier to build and test	Only works under the assumption that the user knows how much free-spending money they have left after

			mandatory bills and spendings

For this design choice, the habit tracker option was chosen as it is aligned more towards the habit tracking narrative.

External API choice for currency conversion

Option	Pros	Cons
Frankfurter	No key, no daily or monthly quota, 223 currencies, caching encouraged and free for commercial use.	Rates aren't fetched live, but rather daily mid-market values
ExchangeRate-API **open access**	No key and has daily updates	Requires a visible attribution link. Rate-limited to roughly one request per hour
ExchangeRate-API free key	No attribution needed	1,500 requests per month. Needs an account and key
Hard-coded rates	No external dependency	Go stale quickly and need manual updates

For currency conversions and exchange rates, Frankfurter is used since it is easier to set up and requires no API key.

LLM choice for AI helper

Option	Notes	Strengths	Drawbacks
Gemini API **(Flash, Flash-Lite)**	Free tier doesn't expire, limits are per project.	Handles large inputs well, useful for large contexts	On the free tier, Google may use inputs and outputs to improve its models, and human reviewers may read them
Groq **(e.g. gpt-oss-120b)**	Rates: 30 requests/min, 1,000/day, 8,000 tokens/min, 200,000 tokens/day	Very fast responses, Generous daily request count and card needed	The tokens-per-minute cap limits how many long, history-heavy chats can run at once.
OpenRouter	Rates: 20 requests/min. 50/day, rising to 1,000/day after a one-off $10 credit purchase	Many models behind one key	Free models rotate out with little notice, and has the lowest daily cap

For setting up the AI chatbot, Groq would be chosen as the preliminary AI of choice as I already have a setup ready for it. However, in the future there may be an automatic feature for swapping between models when the previous one has exhausted all of its tokens.

6.4 Project Timeline

Week	Features
11	1. Setup a. Supabase database setup b. GitHub repositories and commit rulesets c. Google OAuth setup d. Login, registration and logout 2. Core features a. Monthly allowance and daily budget (2.1) b. Income distribution (2.3) c. Spending states (2.7) d. Currency conversion (2.10) e. Mobile layout (2.11)
12	1. Additional features a. Wishlist (2.2) b. AI helper (2.8)
Post-Week Improvements	1. Additional features a. Statistics (2.4) b. Accountability buddy system (2.5) c. Streaks (2.6) d. Notifications